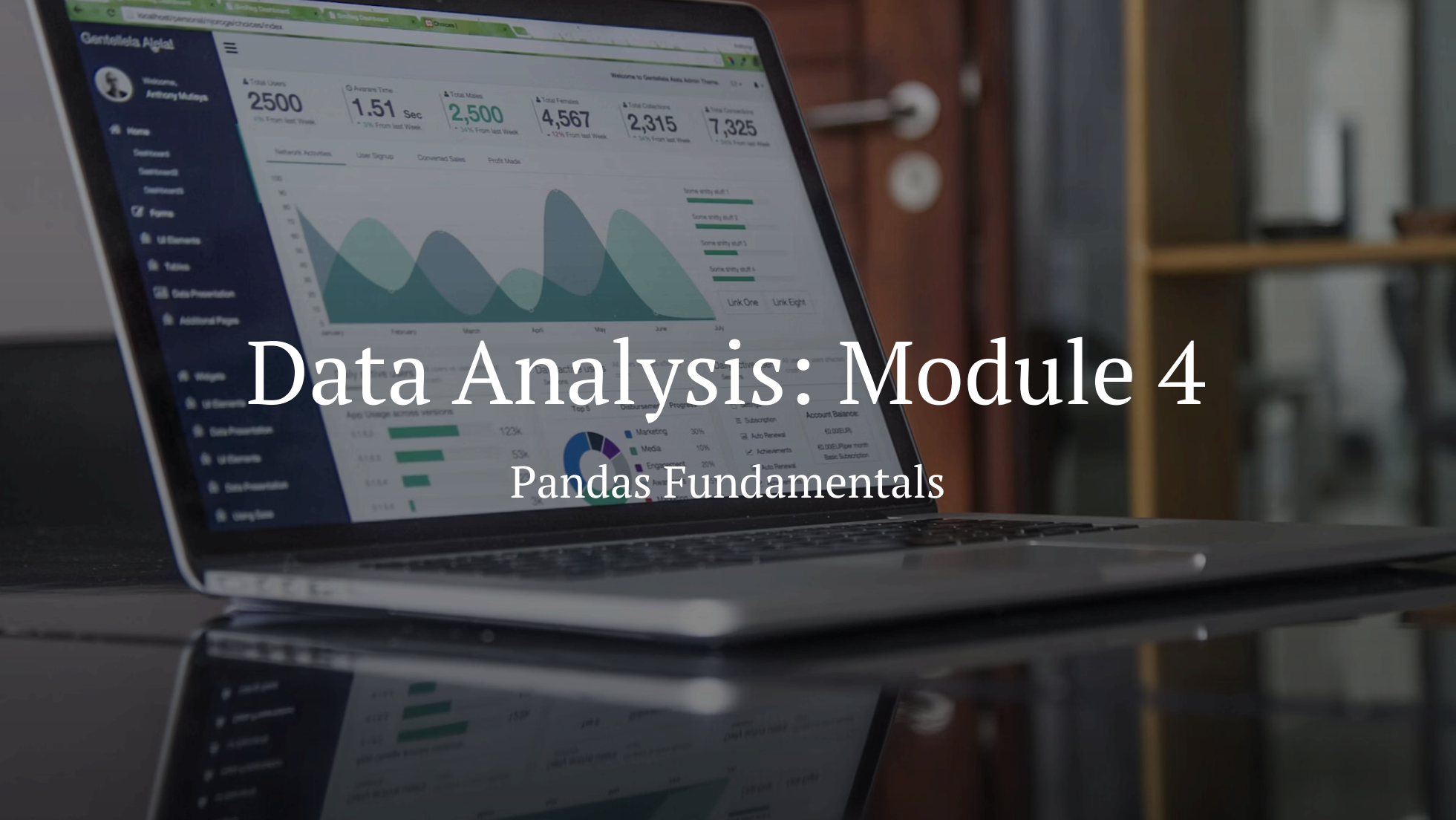

Data Analysis: Module 4

Pandas Fundamentals

1. What is Pandas?

Pandas is a fast, powerful, and flexible open-source data analysis and manipulation tool built on top of the Python programming language.

- **Series:** A 1D labeled array capable of holding any data type.
- **DataFrame:** A 2D labeled data structure with columns of potentially different types (like a spreadsheet or SQL table).

2. Why Use Pandas?

While NumPy is great for numerical data, Pandas excels at handling **tabular** and **heterogeneous** data.

1. **Handling Missing Data:** Easy detection and filling of `NaN` values.
2. **Size Mutability:** Columns can be inserted and deleted from DataFrames.
3. **Powerful Grouping:** "Split-apply-combine" operations for data aggregation.
4. **Time Series Support:** Robust tools for working with dates and frequencies.

If NumPy is the engine... Pandas is the entire luxury dashboard for your data!

3. Creating Data Structures

Series

```python
import pandas as pd

# From a list
s = pd.Series([1, 3, 5, 7])

# With custom index
s = pd.Series([10, 20],
              index=['a', 'b'])
```

DataFrame

```python
# From a dictionary
data = {
    'Name': ['Alice', 'Bob'],
    'Age': [25, 30]
}
df = pd.DataFrame(data)

# From CSV
# df = pd.read_csv('data.csv')
```

4. Viewing & Inspecting Data

Before analyzing, you must understand the "shape" and "health" of your dataset.

```python
import pandas as pd
df = pd.DataFrame({'A': range(100), 'B': range(100)})

# View first 5 rows
print(df.head())

# Get summary of statistics
print(df.describe())

# Check data types and nulls
print(df.info())
```

5. Selection: loc vs iloc

Pandas provides specific methods to slice data by labels or integer positions.

```
# Create a sample DF
df = pd.DataFrame({'Score': [90, 85, 95]},
                  index=['Math', 'Science', 'History'])

# .loc is LABEL-based
print(df.loc['Math'])

# .iloc is INTEGER-based (position)
print(df.iloc[0])
```

6. Filtering Data

You can filter DataFrames using boolean indexing, similar to NumPy.

```
df = pd.DataFrame({'Name': ['Ann', 'Joe', 'Sia'],
                   'Age': [20, 35, 19]})

# Filter for Age > 20
adults = df[df['Age'] > 20]

# Filter with multiple conditions
young_staff = df[(df['Age'] < 30) & (df['Name'] == 'Ann')]
```

7. Basic Data Cleaning

One of the most common tasks is handling missing or incorrect data.

```
# Drop rows with any missing values
df_clean = df.dropna()

# Fill missing values with a specific value
df_filled = df.fillna(0)

# Rename columns
df = df.rename(columns={'Name': 'Full_Name'})
```

8. Summary Comparison

Feature	NumPy	Pandas
Data Type	Homogeneous (Single type)	Heterogeneous (Multiple types)
Indexing	Integer-based only	Label and Integer-based
Primary Use	Mathematical computations	Data manipulation & Analysis
Structure	Arrays (N-dimensional)	Series (1D) and DataFrames (2D)

End

Reference Sources:

- Pandas Official Getting Started
- Real Python: Pandas Tutorial